

Hoja de trabajo 2

Fabián Morales

De las RNN a la atención

En un seq2seq con RNN el encoder resume toda la oración en un vector de tamaño fijo, que es el cuello de botella: su capacidad no crece con la oración y en oraciones largas los primeros tokens se diluyen. La atención de Bahdanau conserva todos los estados del encoder y en cada paso del decoder los pondera según su relevancia. Bahdanau puntúa con una red aditiva, $v \cdot \tanh(Ws + Uh)$, y Luong con el producto punto $s \cdot h$, más eficiente en GPU porque todas las puntuaciones salen de una sola multiplicación de matrices.

La atención es una búsqueda suave en un diccionario: la consulta se compara con todas las llaves y se devuelve el promedio de los valores ponderado por la softmax de las similitudes. En self-attention Q, K y V son proyecciones de la misma secuencia, y en cross-attention Q viene del decoder y K y V del encoder. Sin recurrencia, el Transformer procesa todas las posiciones en paralelo, conecta cualquier par de tokens en un paso en lugar de n y no multiplica el gradiente por la misma matriz en cada paso, lo que lo desvanece.

La arquitectura Transformer

Si q y k tienen componentes independientes de media 0 y varianza 1, cada $q_i k_i$ tiene varianza 1 y $q \cdot k$, suma de d_k términos, tiene varianza d_k . Sin dividir entre $\sqrt{d_k}$ la softmax se satura en una sola llave y su jacobiano tiende a cero, lo que apaga los gradientes. Con h cabezas de dimensión d_{model}/h cada una proyecta a un subespacio distinto y aprende otra relación, en vez de promediarlo todo en una distribución, y el número de parámetros no cambia, $4d^2 + 4d$.

La self-attention es equivariante a permutaciones, así que no conoce el orden. La codificación sinusoidal suma senos y cosenos de frecuencias geométricas y permite extrapolar, los embeddings aprendidos de BERT y GPT-2 no pasan de la longitud máxima, y RoPE rota q y k para que su producto dependa solo de la distancia relativa. Cada subcapa lleva una conexión residual y una LayerNorm, y la red feed-forward expande cada posición a $4 d_{\text{model}}$ y vuelve. Post-LN normaliza después de sumar la residual y necesita warmup, y Pre-LN normaliza antes de la subcapa y entrena de forma más estable en redes profundas.

La self-attention cuesta $O(n^2d)$ en tiempo y $O(n^2)$ en memoria. FlashAttention calcula la atención exacta por bloques con memoria $O(n)$, pero el tiempo sigue cuadrático y depende de kernels de GPU. La atención por ventanas de Longformer cuesta $O(nw)$ y sacrifica la conexión directa entre tokens lejanos, y la atención lineal cuesta $O(n)$ a cambio de aproximar la softmax con un kernel y perder precisión.

Atención en PyTorch y Hugging Face

En `F.scaled_dot_product_attention`, `attn_mask` bloquea o suma valores a las puntuaciones, `dropout_p` aplica dropout a los pesos, `is_causal` aplica la máscara causal sin construirla y `scale`

reemplaza $1/\sqrt{d}$. `nn.MultiheadAttention` recibe `embed_dim`, `num_heads`, `batch_first`, `kdim` y `vdim`, y su forward acepta `key_padding_mask` para el relleno, `attn_mask` para bloquear pares, `need_weights` y `average_attn_weights`. `nn.TransformerEncoderLayer` es un bloque completo con `norm_first` para elegir Pre-LN, `generate_square_subsequent_mask` crea la máscara causal con $-\inf$, y `output_attentions` de Hugging Face devuelve un tensor por capa de lote $\times$ cabezas $\times$ n $\times$ n.

Ejercicio a mano y verificaciones

Con $d_k = 2$ se usaron $Q = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 1, 1 \end{bmatrix}$, $K = \begin{bmatrix} 1, 0 \\ 1, 1 \\ 0, 1 \end{bmatrix}$ y $V = \begin{bmatrix} 1, 0 \\ 0, 2 \\ 3, 1 \end{bmatrix}$. Para el primer token $q \cdot k$ da 1, 1 y 0, que entre $\sqrt{2}$ son 0.7071, 0.7071 y 0, la softmax da 0.4011, 0.4011 y 0.1978, y la salida es $0.4011 \cdot [1, 0] + 0.4011 \cdot [0, 2] + 0.1978 \cdot [3, 1] = [0.9944, 1.0000]$.

Tabla 1. Ejercicio de atención a mano

Token	$QK^T / \sqrt{2}$	Pesos de la softmax	Salida
1	0.7071, 0.7071, 0	0.4011, 0.4011, 0.1978	0.9944, 1.0000
2	0, 0.7071, 0.7071	0.1978, 0.4011, 0.4011	1.4011, 1.2033
3	0.7071, 1.4142, 0.7071	0.2483, 0.5035, 0.2483	0.9930, 1.2552

La atención propia coincide con `F.scaled_dot_product_attention` sin máscara, con máscara causal y con relleno, con error máximo de 4.8×10^{-7} , y la multi-cabeza propia coincide con `nn.MultiheadAttention` con error de 1.2×10^{-7} y reproduce exactamente los pesos de la primera capa de BERT. Las filas suman 1, la máscara causal deja ceros sobre la diagonal, permutar la entrada sin posiciones solo permuta la salida, y 1 cabeza de 768 o 12 de 64 tienen 2,362,368 parámetros.

Escalamiento y costo computacional

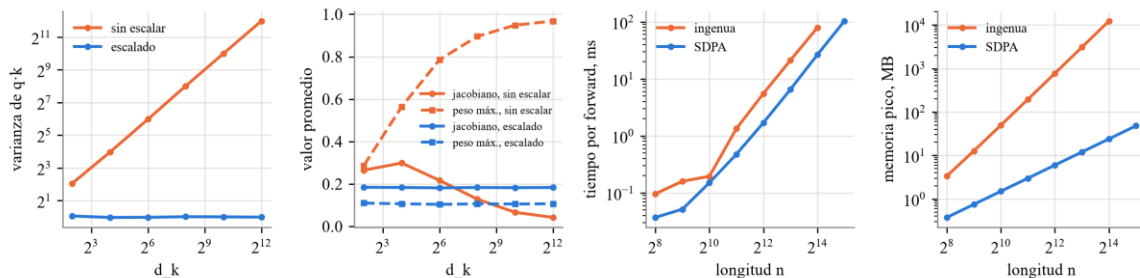


Figura 1. Escalamiento y costo de atención

La varianza de $q \cdot k$ fue 4.1, 62.8 y 4,052 con d_k de 4, 64 y 4,096, y cerca de 1 al escalar. Sin escalar, con d_k de 4,096 la softmax da 0.97 del peso a una sola llave y la norma de su jacobiano cae a 0.04, frente a 0.18 escalada. En la RTX 5070 Ti, con 12 cabezas de 64 en float16, la atención ingenua crece con pendiente log-log de 1.96 en tiempo y 1.99 en memoria y usa 12.3 GB con 16,384 tokens. SDPA, con un kernel tipo FlashAttention, sigue cuadrática en tiempo, con pendiente 1.95, pero es 3 veces más rápida y su memoria crece de forma lineal, 48 MB con 32,768 tokens.

Mapas de atención en BERT y GPT-2

Se midieron seis patrones en las 144 cabezas de cada modelo sobre 8 oraciones. La figura 2 muestra la cabeza más fuerte de cuatro patrones en capas distintas, con c la capa y h la cabeza: en BERT, token anterior, siguiente con 0.93 de peso medio y [SEP] con 0.87, y en GPT-2, token anterior con 0.999, primer token con 0.99, comas y la misma palabra.

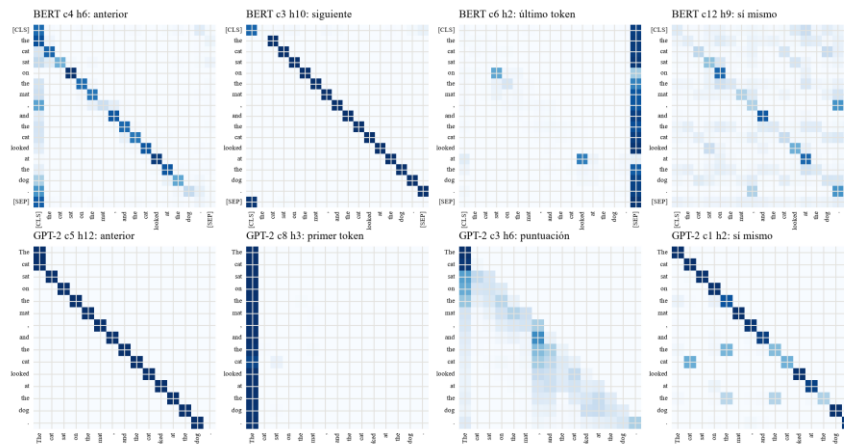


Figura 2. Cabezas de BERT y GPT-2

En el par de oraciones, 22 de las 144 cabezas de BERT hacen que it atienda más a animal con tired y más a street con wide. En la c7 h11, it da 0.28 a animal y 0.07 a street con tired, y 0.59 a street y 0.01 a animal con wide. En GPT-2 la fila de it es idéntica en ambas oraciones porque la máscara causal oculta el adjetivo.

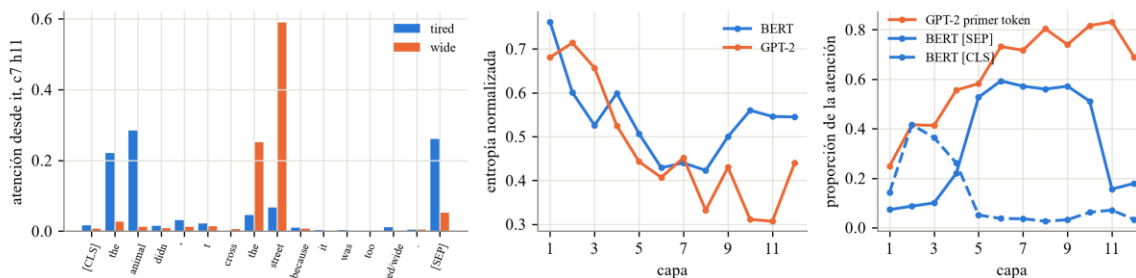


Figura 3. Correferencia, entropía y sumideros

Todas las matrices de GPT-2 son triangulares inferiores, con peso 0 sobre la diagonal. El primer token recibe el 25 % de la atención en la capa 1 y entre 69 y 83 % desde la capa 6, y 104 de las 144 cabezas le dan más de la mitad. Es el sumidero de atención de Xiao et al.: como la softmax obliga a repartir todo el peso, las cabezas sin nada útil que leer lo descargan en el primer token, y anteponer un salto de línea casi no cambia las proporciones. BERT hace lo mismo con [SEP] entre las capas 5 y 10. La entropía normalizada baja con la profundidad, en BERT de 0.76 a 0.42 en la capa 8 y en GPT-2 de 0.71 a 0.31 en las capas 10 y 11, así que las capas profundas atienden de forma más concentrada, en buena parte sobre los sumideros.

Discusión

La atención permitió modelos más grandes porque calcula todas las posiciones en paralelo y conecta dos tokens en un paso en lugar de n , a cambio del costo cuadrático en n .

Las cabezas se especializan en patrones claros, pero solo 22 de 144 distinguen la correferencia de *it*, y 44 cabezas de BERT y 104 de GPT-2 dan más de la mitad de su atención a un sumidero, en línea con Michel et al., que podaron la mayoría de las cabezas con poca pérdida.

Los pesos de atención no bastan como explicación: Jain y Wallace mostraron que pesos muy distintos pueden dar la misma predicción, y Wiegreffe y Pinter que solo son plausibles si se prueban contra alternativas. Aquí los sumideros reciben casi todo el peso sin aportar contenido.

Una RNN o una CNN siguen siendo preferibles en flujos en tiempo real, con hardware limitado o pocos datos. Con 100,000 tokens la matriz de atención tendría 10^{10} elementos, unos 20 GB por cabeza y capa en float16, así que se usaría FlashAttention, atención por ventanas o lineal, modelos de espacio de estados o recuperación.

Conclusiones

1. La atención propia reproduce exactamente a PyTorch y a BERT, y sin dividir entre $\sqrt{d_k}$ la varianza crece como d_k y la softmax se satura.
2. Las cabezas se especializan y algunas resuelven la correferencia, pero gran parte de la atención profunda va a sumideros como el primer token o [SEP].
3. El costo cuadrático limita la atención: la versión ingenua usa 12.3 GB con 16,384 tokens y SDPA reduce la memoria a crecimiento lineal.

Referencias

- Brenndoerfer, M. (2026, 11 de febrero). Attention visualization: Extracting and interpreting weights. <https://mbrenndoerfer.com/writing/attention-visualization>
- Cho, A., Kim, G. C., Karpekov, A., Helbling, A., Wang, Z. J., Lee, S., Hoover, B. y Chau, D. H. (s. f.). Transformer Explainer: LLM Transformer model visually explained. Georgia Institute of Technology. Recuperado el 5 de octubre de 2026, de <https://poloclub.github.io/transformer-explainer/>
- Guessous, D. (2024, 9 de octubre). (Beta) Implementing high-performance transformers with scaled dot product attention (SDPA). PyTorch Tutorials. https://docs.pytorch.org/tutorials/intermediate/scaled_dot_product_attention_tutorial.html
- Liu, Y., Yu, J., Xu, Y., Li, Z. y Zhu, Q. (2025). A survey on transformer context extension: Approaches and evaluation (Versión 2) [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2503.13299>

Repositorio: <https://github.com/FabianMoraeles/DeepHoja2>